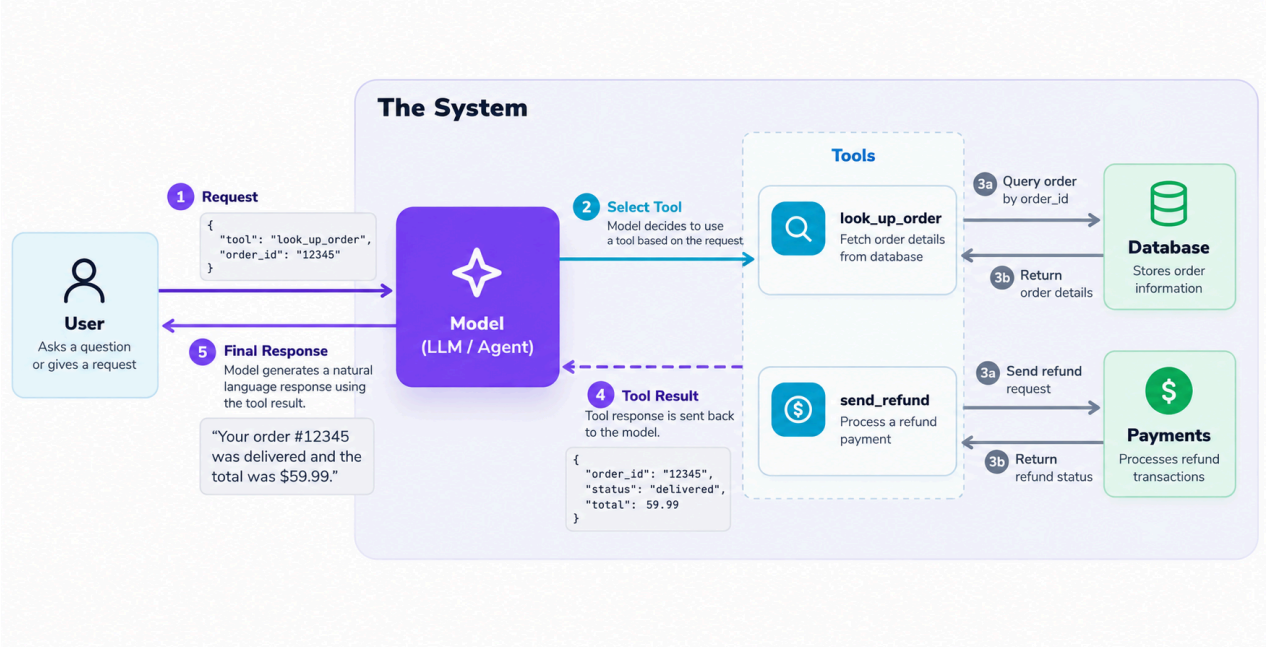
# Module 1: Agent & Chatbots

Agent vs chatbot
**Chatbot** - can only respond to messages
**Agent** - Responds to the messages and acts upon the user responses.

Example:  If a user gets broken order and wants a refund
**Chatbot** - will only give user refund policies but does not actually refunds the user.
**Agent** - will talk to the user , give user refund policies, check the refund policies and the days the refund should be initiated within and initiate the refund to the user.

**LLM** - itself doesn't interact with external tools to initiate the refund.
System around the model handles the calling part i.e **Tools**

**Tools -** It's a piece of software that lets a model talk to.




This is called the Agentic Loop. An agent

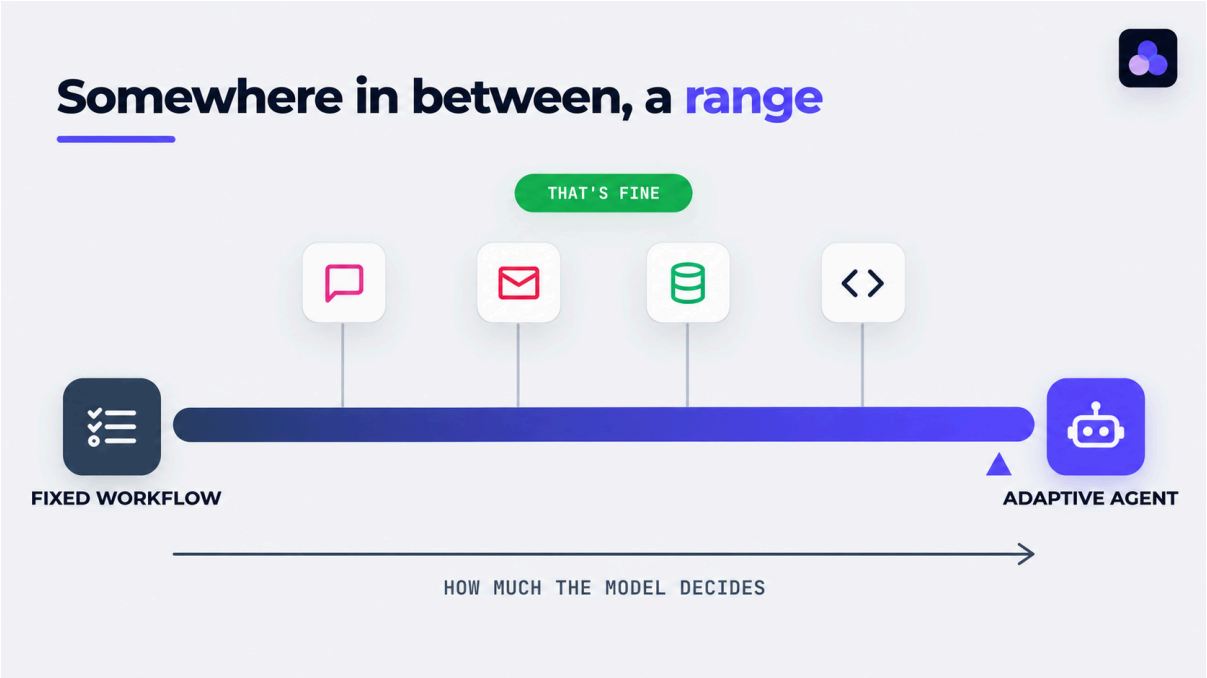
1. DECIDE
2. ACT
3. LOOK AT THE RESULT

**ReAct**- Reasoning with actions

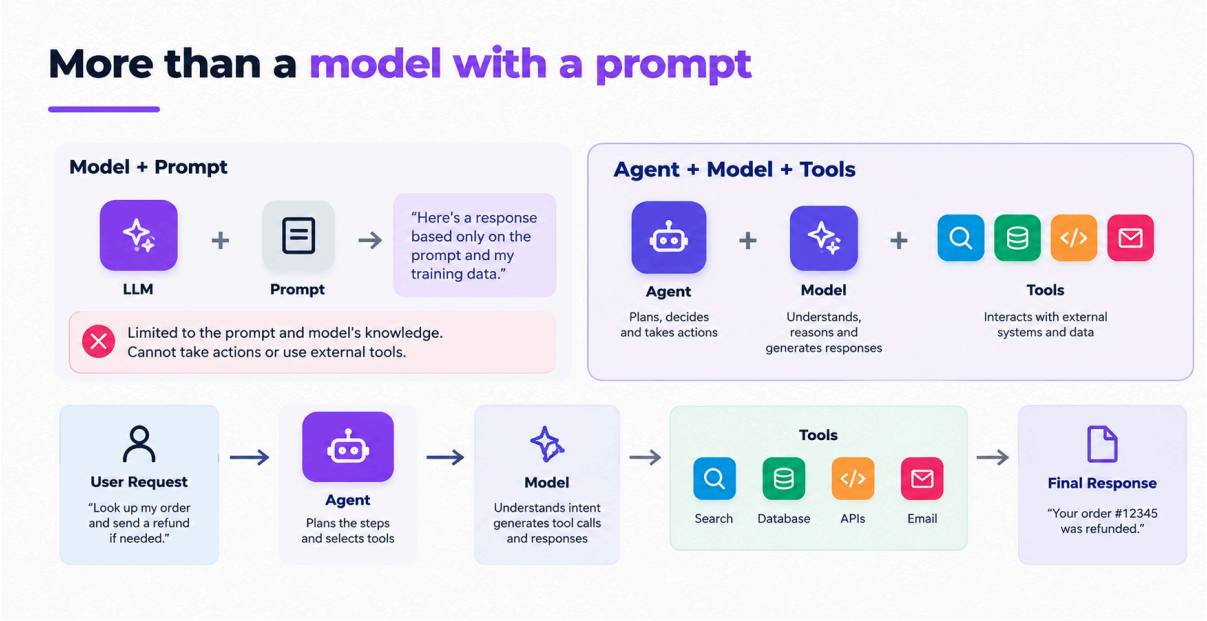
Model reasons about what to do and take action and look at the result and reasons again.

One Distinction:

Workflow vs Agent
Workflow - when a person writes the steps ahead of time and then the model fills each step .
Agent - When the model gets to decide what to do next instead of a human telling it what to do ahead .




The more the agent is adaptive, the better it is , but it is always good to limit the number of steps the agent can take. At the end of the day it is good to have human interference for correctness.




Capabilities:

1. Perception
2. Reasoning
3. Action
4. Learning
5. Communication

Not that every agent must have this but these are the things we should look for.

1 agent ≠ 1 model
An Agent can call multiple models and can divide the task.

How does a model remember the context ?

1. Short Term memory- recent info agent keep in current context.
2. Episodic Memory - record of what happened and when - useful in fail scenario
3. Semantic memory - Facts(Knowledge Graph)
4. Procedural Memory- knows the procedure (ex: Refunding a customer)

**Memory vs Context**
Memory - what stored
Context - What get put together for one model call

**Context Window:** It is working space for one model call

- Instructions
- Conversations so far
- Documents you uploaded
- Tool Results

It is measured in tokens . Every model has a limit on how many tokens fit.
Researchers found that in a model context window . The model makes better use of start info and end info of the context window rather than middle- The cause is still unknown.

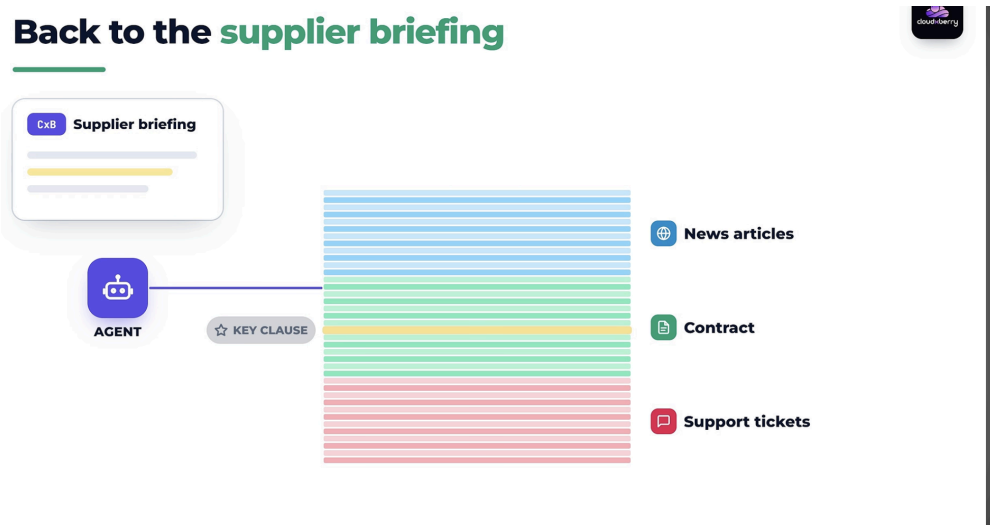



A long context doesn't necessarily guarantee that mode will use each piece of the information equally well.

Large Context ->More Computation->More waiting ->More Cost

As the context grows , key details could be harder for the model to use.

A bigger context window doesn't solve the problem.It means the model will fit in more information but doesn't guarantee that it takes in key detail for sure. Position of the key document or key info always matters




How to deal with it?

1. Keep only what's needed
2. Keep in order - Positioning of key detail is important